A PROJECT REPORT ON

REALTIME SIGN-TO-TEXT CONVERTER

SUBMITTED TO
MIT SCHOOL OF COMPUTING, LONI, PUNE IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE

BACHELOR OF TECHNOLOGY
Computer Science & Engineering

BY

Rudra Shelke – ADT23SOCB0886
Samruddhi Kaule – ADT23SOCB0961
Samarthi Mante – ADT23SOCB1404
Pavan Memane – ADT23SOCB0712

Under the guidance of

Prof. Viddulata Patil



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MIT School OF COMPUTING
MIT Art, Design and Technology University
Rajbaug Campus, Loni-Kalbhor, Pune 412201

2025- 26

MIT SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY,
RAJBAUG CAMPUS, LONI-KALBHOR, PUNE 412201

CERTIFICATE

This is to certify that the project report entitled

"REALTIME SIGN TO TEXT CONVERTER"

Submitted by

Name of Candidate: Rudra Shelke	EnrollmentNo: ADT23SOCB0886
Name of Candidate: Samruddhi Kaule	EnrollmentNo.: ADT23SOCB0961
Name of Candidate: Samarthi Mante	EnrollmentNo.: ADT23SOCB1404
Name of Candidate: Pavan Memane	EnrollmentNo.: ADT23SOCB0712

is a bonafide work carried out by them under the supervision of Prof. Viddulata Patil and it is submitted towards the partial fulfillment of the requirement of MIT ADT university, Pune for the award of the degree of Bachelor of Technology (Computer Science and Engineering)

Prof. Viddulata Patil
Guide

Dr.Nandakumar Kulkarni
Head of Department

Prof. Suresh Kapare
Chief Coordinator-PBL

Dr. Ganesh Pathak
Dean

Seal/Stamp of the College
Place: PUNE
Date: 24th April 2026

DECLARATION

We, the team members

Name	Enrollment No
Rudra Shelke	ADT23SOCB0886
Samruddhi Kaule	ADT23SOCB0961
Samarthi Mante	ADT23SOCB1404
Pavan Memane	ADT23SOCB0712

Hereby declare that the project work incorporated in the present project entitled **"Realtime Sign To Text Converter"** is original work. This work (in part or in full) has not been submitted to any University for the award or a Degree or a Diploma. We have properly acknowledged the material collected from secondary sources wherever required. We solely own the responsibility for the originality of the entire content.

Date: 24/04/2026

Name & Signature of the Team Members

Member 1: Rudra Shelke

Member 2: Samruddhi Kaule

Member 3: Samarthi Mante

Member 4: Pavan Memane

Name and Signature of Guide

Prof. Viddulata Patil

Seal/Stamp of the College

Place: Pune

Date: 24/04/2026



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MIT SCHOOL OF COMPUTING,
RAJBAUG, LONI KALBHOR,
PUNE – 412201

EXAMINER'S APPROVAL CERTIFICATE

The project report entitled "**Realtime Sign to Text Converter**" submitted by Rudra Shelke (ADT23SOBC0886), Samruddhi Kaule (ADT23SOCB0961), Samarthi Mante (ADT23SOCB1404), Pavan Memane (ADT23SOCB0712) in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science & Engineering) during the academic year 2025-26, of **MIT-ADT University, MIT School OF COMPUTING, Pune**, is hereby approved.

Examiners:

1.

2.

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to all those who have supported and guided us throughout the completion of our **Project-Based Learning (PBL)** project titled "**Realtime Sign-to-Text Converter**" during the academic year 2025–26.

We would like to extend our heartfelt thanks to our guide, Prof. Viddulata Patil, for her valuable guidance, constant encouragement, and continuous support throughout the development of this project. Her expertise and insights have been instrumental in helping us achieve our objectives successfully.

We also express our deep gratitude to the **Department of Computer Engineering, MIT Art, Design and Technology University, Pune,** for providing us with the facilities and resources required for this work. We are equally thankful to our faculty members and laboratory staff for their cooperation and timely assistance.

Project Team:

1. Rudra Shelke
2. Samruddhi Kaule
3. Samarthi Mante
4. Pavan Memane

Guide: Prof. Viddulata Patil

Academic Year: 2025–2026

ABSTRACT

The Real-Time Sign-to-Text Converter is an AI-based system designed to bridge the communication gap between the hearing and speech-impaired community and the general population. The system captures hand gestures and sign language alphabets in real time using a standard camera and converts them into readable text, enabling seamless communication.

It utilizes MediaPipe Hands for extracting hand landmarks and a Random Forest model for accurate gesture and alphabet classification. The system recognizes both predefined gestures and sign language alphabets (A–Z and digits 0–9), providing real-time predictions with high accuracy and low latency. A word formation module is also implemented to combine detected alphabets into meaningful text.

Unlike traditional systems that rely on specialized hardware, this approach is visionbased, cost-effective, and deployable on standard devices. The application is built using Streamlit and supports cross-platform usage on both desktop and mobile devices, making it accessible and user-friendly.

CONTENTS

Certificate i

Declaration ii

Examiner's Approval Certificate iii

Acknowledgement iv

Abstract v

List of Figures vi

List of Tables vii

Chapter 1: INTRODUCTION 1

1.1 Introduction 1

1.2 Existing Work 3

1.3 Motivation 5

1.4 Objectives 6

1.5 Scope 7

Chapter 2: CONCEPTS AND METHODS 9

2.1 Definitions 9

Chapter 3: LITERATURE SURVEY 12

3.1 Introduction 12

3.2 Review of Existing Systems 13

3.3 Comparative Analysis 16

3.4 Research Gap 18

Chapter 4: PROJECT PLAN 20

4.1 Project Development Model 20

4.1.1 Model Selection 20

4.1.2 Stages of Development 21

4.2 Project Workflow 23

4.3 Timeline and Milestones 25

4.4 Resource Utilization 26

Chapter 5: SOFTWARE REQUIREMENT SPECIFICATION ...28

5.1 Project Scope 28

5.2 User Classes & Characteristics 29

5.3 Functional Requirements 30

5.4 Non-Functional Requirements 31

5.5 Software Requirements 32

5.6 Hardware Requirements 32

5.7 System Constraints 33

5.8 Assumptions 33

Chapter 6: RESULTS 35

Chapter 7: SOFTWARE TESTING 40

7.1 Testing Methodology 40

7.2 Testing Results 41

Chapter 8: CONCLUSION AND FUTURE WORK 43

8.1 Conclusion 43

8.2 Future Work 44

BIBLIOGRAPHY 45

ANNEXURE A: List of Publications and Research Papers

(In its Original Formats) 47

ANNEXURE B: Plagiarism Report 48

ANNEXURE C: Empathy Chart and Solution Document 49

ANNEXURE D: Project Tracker 50

LIST OF FIGURES

Figure No.	Title	Page
Fig 1.1	System Overview	6
Fig 1.2	System User Interface	8
Fig 2.1	Hand Landmark Detection using MediaPipe (21 Key Points)	10
Fig 4.1	System Architecture of Sign-to-Text Converter	21
Fig 4.2	Flowchart of System Workflow	23
Fig 5.1	Use Case Diagram of Sign-to-Text Converter System	29
Fig 5.2	Functional Block Diagram of the System	30
Fig 6.1	Gesture Recognition for Thumbs Up	35
Fig 6.2	Gesture Recognition System for Peace	35
Fig 6.3	Alphabet Prediction by App Deployed on Android	36
Fig 6.4	Complete Word Formed Using Alphabet Recognition	36
Fig 6.5	Overall Performance Metrics of the System	37
Fig C.1	Empathy Map of Sign-to-Text Converter System	49

LIST OF TABLES

Table No.	Title
Table 1.1	Comparative Analysis of Existing Systems
Table 3.1	Literature Survey Comparative Analysis
Table 4.1	Timeline and Milestones
Table 6.1	Performance Summary
Table 7.1	Sample Test Cases and Results
Table 7.2	System Performance under Different Conditions

CHAPTER 1 INTRODUCTION

1.1 Introduction

The Real-Time Sign-to-Text Converter is an AI-based system designed to bridge the communication gap between the hearing and speech-impaired community and the general population. The system captures hand gestures and sign language alphabets in real time using a standard camera and converts them into readable text, enabling seamless and inclusive communication.

With the rapid advancement in computer vision and artificial intelligence, human-computer interaction is becoming more natural and intuitive. Hand gesture recognition has emerged as an effective method for interaction, especially in assistive technologies. This project combines gesture recognition and alphabet recognition to create a unified system capable of understanding both simple gestures and structured sign language inputs.

The system utilizes MediaPipe Hands to detect 21 hand landmarks, which are used to represent the position and orientation of the hand. These landmarks are processed using normalization techniques and classified using a Random Forest model to accurately recognize gestures and alphabets (A–Z and digits 0–9). The system operates in real time with high accuracy and low latency, ensuring smooth performance.

Unlike traditional systems that depend on sensor-based gloves or specialized hardware, this approach is purely vision-based, cost-effective, and easily deployable on standard devices. Additionally, a word formation mechanism is implemented to combine recognized alphabets into meaningful text, enhancing usability.

Overall, the system provides a scalable and efficient solution for real-time sign language interpretation, contributing towards accessible and inclusive communication.

1.2 Existing Work

There are several existing systems related to gesture recognition and sign language interpretation. After analyzing different approaches, we observed that these systems use a variety of techniques such as sensor-based hardware, computer vision, and deep learning models. However, each of these methods has certain limitations in terms of cost, accuracy, or real-time performance.

The following table provides a comparative analysis of some existing systems:

Sr. No.	System/Approach	Method Used	Advantages	Limitations
1	Sensor-Based Gloves	Accelerometers, Sensors	High accuracy	Expensive,not user-friendly
2	Kinect-Based Systems	Depth Camera	Good 3D tracking	Requires special hardware
3	CNN-Based Models	Deep Learning	High accuracy	Requires large dataset and GPU
4	Image Processing Systems	OpenCV Techniques	Simple implementation	Low accuracy in Complex backgrounds
5	Vision-Based Systems	MediaPipe + ML	Real-time, low cost	Sensitive to lighting

Table 1.1 Comparative Analysis of Existing Systems

Most traditional systems rely on hardware-based solutions such as gloves or depth condition sensors, which increase the cost and reduce portability. On the other hand, deep learning-based approaches provide high accuracy but require large datasets and high computational resources, making them less suitable for real-time applications on standard devices.

1.3 Motivation

Communication plays a vital role in everyday human interaction. However, individuals who rely on sign language often face difficulties in communicating with people who do not understand it. This creates a communication barrier that limits their ability to express themselves effectively in society.

With the advancement of technology, there is a growing need for systems that can enable natural and seamless interaction between humans and machines. Gesture recognition and sign language interpretation are important areas in assistive technology that can help bridge this gap. However, many existing systems are either expensive, require specialized hardware, or are not efficient enough for real-time use.

The motivation behind this project is to develop a simple, cost-effective, and real-time solution that can translate hand gestures and sign language alphabets into readable text using a standard camera. By combining gesture recognition with alphabet recognition and word formation, the system aims to provide a more practical and user-friendly approach for communication.

This project also focuses on improving accessibility by creating a platform-independent solution that can run on both desktop and mobile devices. Ultimately, the goal is to contribute towards inclusive technology that supports better interaction and communication for the hearing and speech-impaired community.

1.4 Objectives

The primary objectives of this project are:

- To develop a real-time sign-to-text conversion system using a standard camera without requiring specialized hardware.
- To implement an efficient hand gesture and alphabet recognition system using MediaPipe and machine learning techniques.
- To accurately recognize sign language alphabets (A–Z) and digits (0–9) using hand landmark features.
- To achieve high accuracy and low latency for real-time performance on standard computing devices.
- To design a word formation mechanism that combines recognized alphabets into meaningful text.

- To build a user-friendly and accessible interface using Streamlit for seamless interaction.
- To ensure cross-platform compatibility, enabling the system to run on both desktop and mobile devices.
- To create a cost-effective and scalable solution for assistive communication.

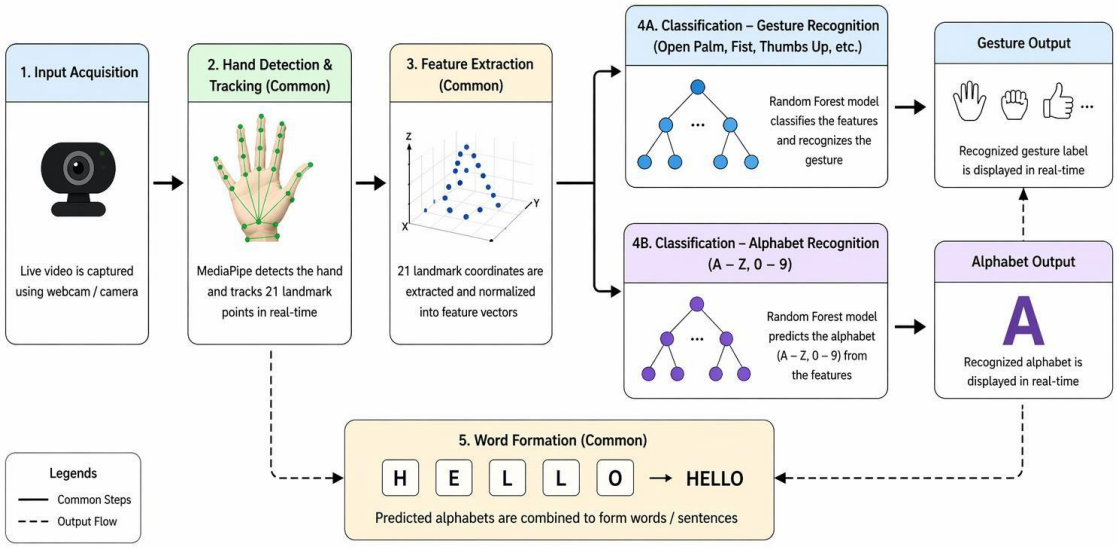


Fig 1.1 System Overview

1.5 Scope

The proposed system focuses on developing a real-time sign-to-text conversion solution using computer vision and machine learning techniques. The scope of the project includes the recognition of hand gestures and sign language alphabets using a standard camera without the need for specialized hardware.

The system is capable of detecting and interpreting single-hand gestures and alphabets (A–Z and digits 0–9) in real time. It also includes a word formation mechanism that combines recognized alphabets into meaningful text, improving the practical usability of the system.

The application is designed to run on standard devices such as laptops and smartphones through a web-based interface, ensuring accessibility and portability. The system performs effectively under normal lighting conditions and controlled backgrounds.

However, the current scope is limited to static gestures and single-hand recognition. Dynamic gestures, sentence-level interpretation, and multi-hand interactions are not included in the present implementation and can be considered for future enhancements.

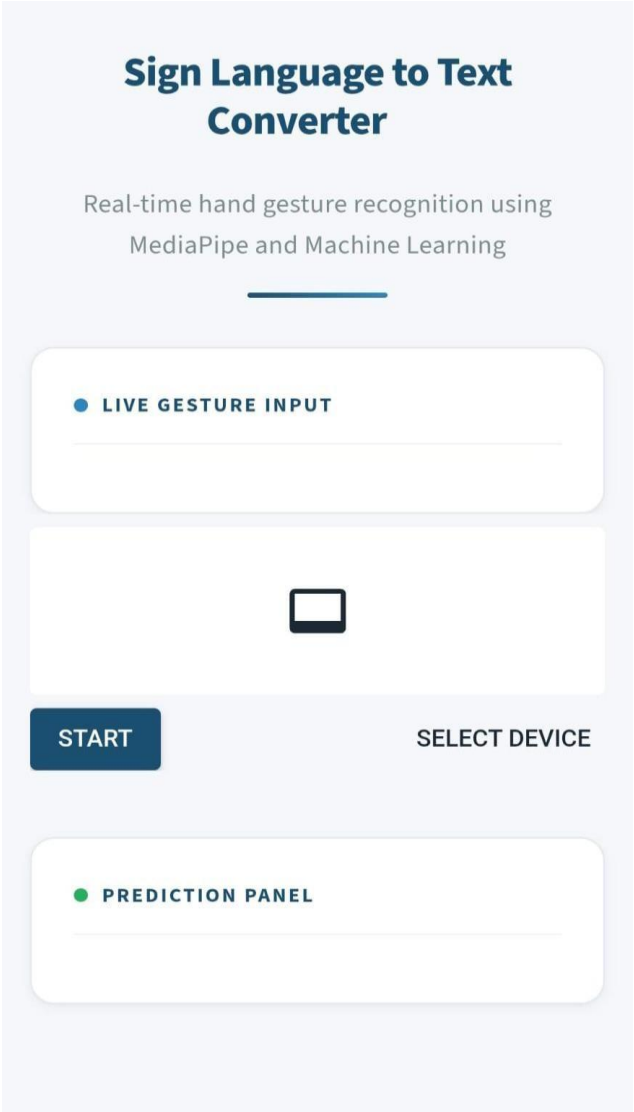


Fig 1.2 System User Interface

CHAPTER 2 CONCEPTS AND METHODS

2.1 Definitions

1. **MediaPipe Hands:**

   MediaPipe Hands is a framework developed by Google for real-time hand tracking. It detects 21 hand landmarks from a single image or video frame, which are used to represent the position and movement of the hand. In this project, it is used for accurate hand detection and feature extraction.

2. **OpenCV:**

   OpenCV (Open Source Computer Vision Library) is used for image and video processing. It helps in capturing video frames, converting color formats, and displaying the output with predictions in real time.

3. **Random Forest Classifier:**

   Random Forest is a machine learning algorithm used for classification tasks. It works by creating multiple decision trees and combining their outputs to improve accuracy. In this system, it is used to classify hand gestures and alphabets based on extracted landmark features.

4. **Hand Landmarks:**

   Hand landmarks are specific key points on the hand such as fingertips and joints. MediaPipe detects 21 such points, which are used to form a feature vector for gesture and alphabet recognition.

5. **Normalization:**

   Normalization is a preprocessing technique used to make the model independent of hand position and size. The coordinates of landmarks are adjusted relative to the wrist and scaled to ensure consistent input for the model.

6. **Streamlit:**

   Streamlit is a Python-based framework used to build interactive web applications. In this project, it is used to create a user interface for displaying real-time predictions and enabling user interaction.

7. **WebRTC:**

   WebRTC (Web Real-Time Communication) is used for capturing and streaming video directly through the web browser. It enables real-time communication between the camera and the application without significant delay.

8. Gesture Recognition:

Gesture recognition refers to the process of identifying hand movements or positions and interpreting them as specific commands or inputs.

9. Alphabet Recognition:

Alphabet recognition is the process of identifying hand signs corresponding to alphabets (A–Z) and digits (0–9), which are then converted into text output.

10. Word Formation:

Word formation is the process of combining recognized alphabets over time to form meaningful words, improving the usability of the system.

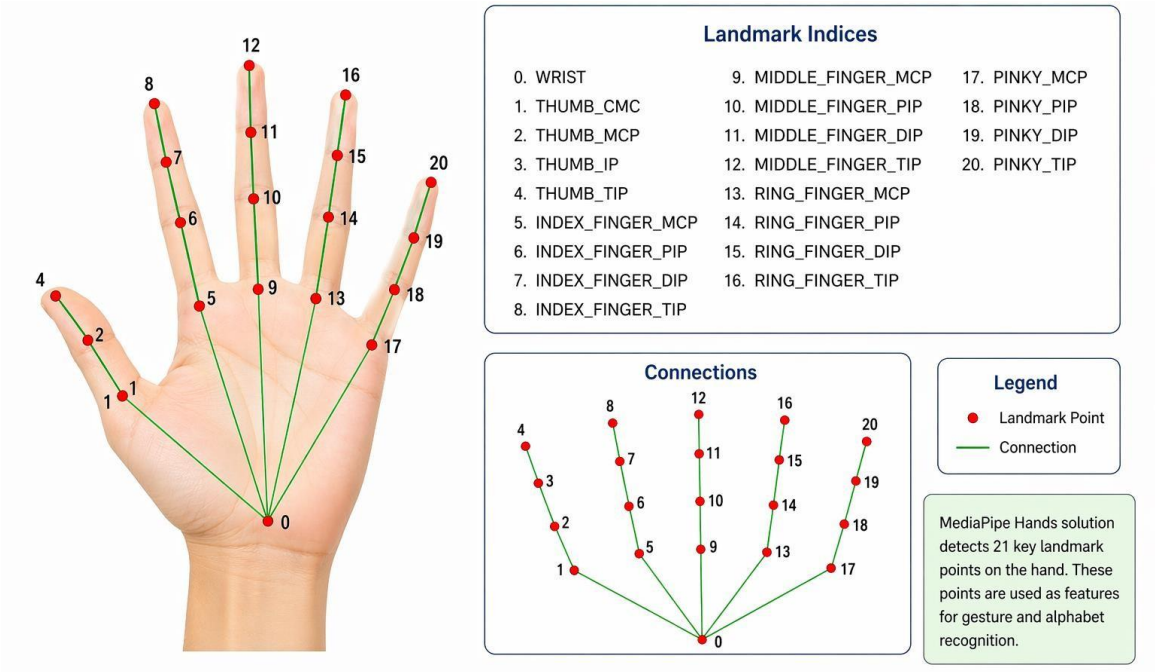


Fig 2.1: Hand Landmark Detection using MediaPipe (21 Key Points)

Chapter 3 LITERATURE SURVEY

3.1 Introduction

Research in gesture recognition and sign language interpretation has evolved significantly due to advancements in computer vision and machine learning. The primary aim of such systems is to interpret hand gestures and convert them into meaningful outputs such as text, thereby facilitating communication for individuals with hearing or speech impairments.

The proposed system — *Real-Time Sign-to-Text Converter using Hand Landmarks* — builds upon prior work in gesture recognition and combines both gesture detection and alphabet recognition into a unified framework. It focuses on providing a practical, lowcost, and real-time solution using lightweight technologies such as MediaPipe, OpenCV, and machine learning models.

3.2 Review of Existing Systems

1. Bhuyan et al. (2019) – "Vision-Based Sign Language Recognition: A Review," IEEE Access.

Bhuyan and his team conducted a comprehensive survey on vision-based sign language recognition systems. They highlighted the importance of image processing and machine learning techniques in gesture interpretation and emphasized the role of deep learning models for improving accuracy in gesture classification.

2. MediaPipe Hands (Google AI Framework, 2023).

MediaPipe Hands provides a real-time hand tracking solution capable of detecting 21 hand landmarks using a standard RGB camera. This framework significantly improves the efficiency of gesture recognition systems by reducing computational complexity and enabling real-time processing on low-end devices.

3. Random Forest Classification (Breiman, 2001).

Random Forest is an ensemble learning algorithm widely used for classification tasks. It combines multiple decision trees to improve prediction accuracy and reduce overfitting. It is particularly effective for landmark-based classification due to its ability to handle nonlinear feature relationships.

4. OpenCV Documentation(2024).

OpenCV is widely used for real-time image and video processing. It provides functionalities such as frame capture, image transformation, and visualization, which are essential for gesture recognition systems.

5. CNN-Based Gesture Recognition Systems (Zhang et al., 2021).

This approach uses Convolutional Neural Networks for gesture classification. CNNs are capable of extracting spatial features automatically and achieving high accuracy. However, they require large datasets and high computational resources.

6. More & Naik (2022) – "Real-Time Indian Sign Language Recognition Using CNN." Their system focused on recognizing Indian Sign Language alphabets using CNN models and achieved around 92% accuracy. It demonstrated that vision-based systems using webcams can provide effective recognition without specialized hardware.

7. Mishra & Patil (2023) – "Real-Time Hand Gesture Recognition using Mediapipe." This study showed that combining MediaPipe landmark extraction with machine learning models improves speed and reduces computational cost while maintaining high accuracy in real-time scenario.

3.3 Comparative Analysis

A comparative summary of related research is presented below:

Author & Year	Technique Used	Framework	Accuracy	Remarks
Bhuyan et al., 2019	Image-based Recognition	Custom Models	90%	Strong theoretical foundation
Zhang et al., 2021	CNN-based	TensorFlow	~95%	High computational cost
More & Naik, 2022	Static ISL Recognition	OpenCV + CNN	~92%	Limited to alphabets
Mishra & Patil, 2023	Landmark-based	MediaPipe	~94%	Efficient and realtime
Proposed System	Landmark + ML	MediaPipe + RandomForest	~95%	Low-cost and real-time

Table 3.1 Literature Survey Comparative Analysis

From the comparative analysis, it is observed that traditional deep learning approaches provide high accuracy but require high computational power. In contrast, landmark-based approaches using MediaPipe offer a more efficient and lightweight solution suitable for real-time applications.

3.4 Research Gap

Despite significant progress in gesture recognition systems, several limitations still exist in current approaches:

- Many systems rely on high computational resources such as GPUs.
- Deep learning models require large datasets for training.
- Several systems are limited to either gesture recognition or alphabet recognition, but not both.
- Performance is often affected by environmental factors such as lighting and background noise.

The proposed system addresses these limitations by:

- Using MediaPipe for lightweight and efficient hand landmark detection.
- Applying a Random Forest classifier for accurate and fast prediction.
- Integrating gesture recognition, alphabet recognition, and word formation into a single system.
- Ensuring real-time performance on standard devices without requiring specialized hardware.

CHAPTER 4 PROJECT PLAN

Developing a real-time AI-based gesture and alphabet recognition system requires a systematic and structured approach that integrates both software engineering principles and machine learning workflows. The *Real-Time Sign-to-Text Converter using Hand Landmarks* project was implemented using a hybrid Waterfall–Agile model to ensure organized development while allowing iterative improvements based on testing and performance evaluation.

4.1 Project Development Model

4.1.1 Model Selection

The project follows a hybrid Waterfall–Agile methodology.

- The **Waterfall approach** ensures a step-by-step development process, starting from requirement analysis to final deployment.
- The **Agile approach** allows flexibility in refining the machine learning model, improving accuracy, and updating the user interface during testing phases.

This combination ensures both structured development and adaptability.

4.1.2 Stages of Development

1. **Requirement Analysis** – Identification of system requirements including gesture types, dataset structure, software tools, and expected outputs.
2. **System Design** – Creation of system architecture, data flow, and design of the gesture and alphabet recognition pipeline.
3. **Data Collection & Preprocessing** – Capturing hand gesture images using a webcam and extracting hand landmarks using MediaPipe.
4. **Feature Extraction & Model Training** – Converting landmark coordinates into normalized feature vectors and training the Random Forest classifier.
5. **System Integration** – Integrating MediaPipe-based hand detection with the trained model and building the user interface using Streamlit.
6. **Testing & Validation** – Performing unit testing and real-time testing to evaluate accuracy, latency, and robustness.
7. **Documentation & Deployment** – Preparing project documentation and deploying the system for demonstration and usage.

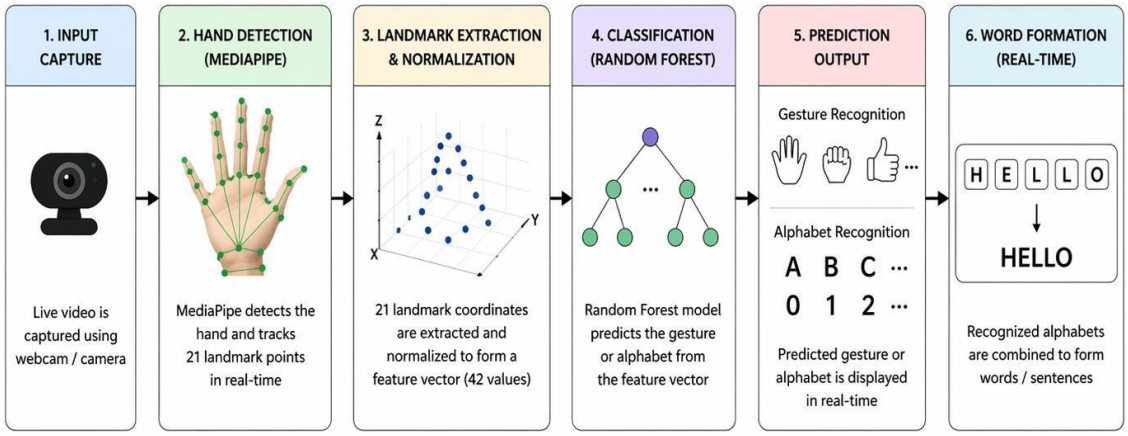


Fig 4.1: System Architecture of Sign-to-Text Converter

4.2 Project Workflow

The overall workflow of the system is described as follows:

1. **Input Acquisition:**
   The system captures live video input from the webcam or mobile camera.
2. **Hand Detection:**
   MediaPipe detects 21 hand landmarks in each frame, representing the hand structure.
3. **Feature Extraction and Preprocessing:**
   The landmark coordinates are normalized to remove variations due to hand position and distance from the camera.
4. **Classification using Machine Learning:**
   The processed features are passed to a trained Random Forest model to classify gestures and alphabets (A–Z and digits 0–9).
5. **Text Generation and Display:**
   The predicted output is displayed as text in the user interface, and alphabets are combined to form meaningful words.

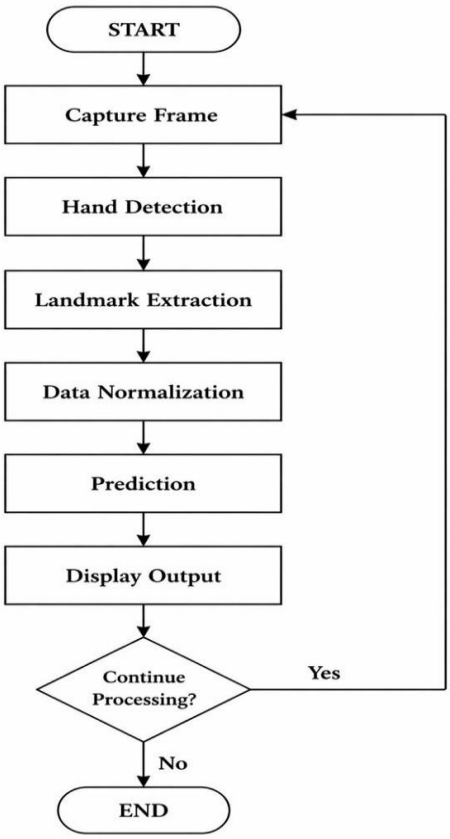


Fig 4.2: Flowchart of System Workflow

4.3 Timeline and Milestones

Phase	Duration	Key Deliverables
Requirement Analysis	1 week	Defined objectives, tools, and system requirements
System Design	1 week	Architecture diagrams and workflow design
Data Collection & Preprocessing	2 weeks	Dataset of gestures and alphabets
Model Training & Evaluation	2 weeks	Trained Random Forest model with high accuracy
Integration & Testing	2 weeks	Fully functional real-time system
Documentation & Final Review	1 week	Final report and project presentation

Table 4.1 Timeline and Milestones

4.4 Resource Utilization Hardware

Resources:

- Laptop with Intel i5 processor and 8GB RAM
- Integrated webcam or mobile camera
- No external sensors required

Software Resources:

- Python 3.x
- MediaPipe (hand landmark detection)
- OpenCV (image processing)
- Scikit-learn (Random Forest model)
- Streamlit (web-based user interface)
- NumPy and Pandas (data processing)

Dataset:

- Custom dataset of hand gestures and alphabets captured using a webcam
- Landmark-based feature dataset generated using MediaPipe
- Data preprocessing and normalization applied for consistency

CHAPTER 5 SOFTWARE REQUIREMENT SPECIFICATION

5.1 Project Scope

The proposed system aims to develop a real-time sign-to-text conversion solution using computer vision and machine learning techniques. The system captures hand gestures and sign language alphabets using a standard camera and converts them into readable text, enabling effective communication.

The system supports recognition of static hand gestures and alphabets (A–Z and digits 0–9). It also includes a word formation mechanism that combines detected alphabets into meaningful text. The application is designed to run on standard devices such as laptops and mobile phones through a web-based interface.

The project focuses on providing a low-cost, accessible, and real-time solution without requiring specialized hardware. However, the current scope is limited to single-hand recognition and static gestures. Dynamic gestures and sentence-level interpretation are considered as future enhancements.

5.2 User Classes & Characteristics

User Classes

General Users

- Individuals who want to interact with the system using hand gestures.
- No prior technical knowledge required.

Hearing/Speech-Impaired Users

- Primary users who rely on sign language for communication.
- Use the system to convert gestures into readable text.

User Characteristics

- Basic understanding of using a smartphone or computer
- Ability to perform hand gestures clearly in front of a camera
- No need for technical or programming knowledge
- Requires a device with camera access

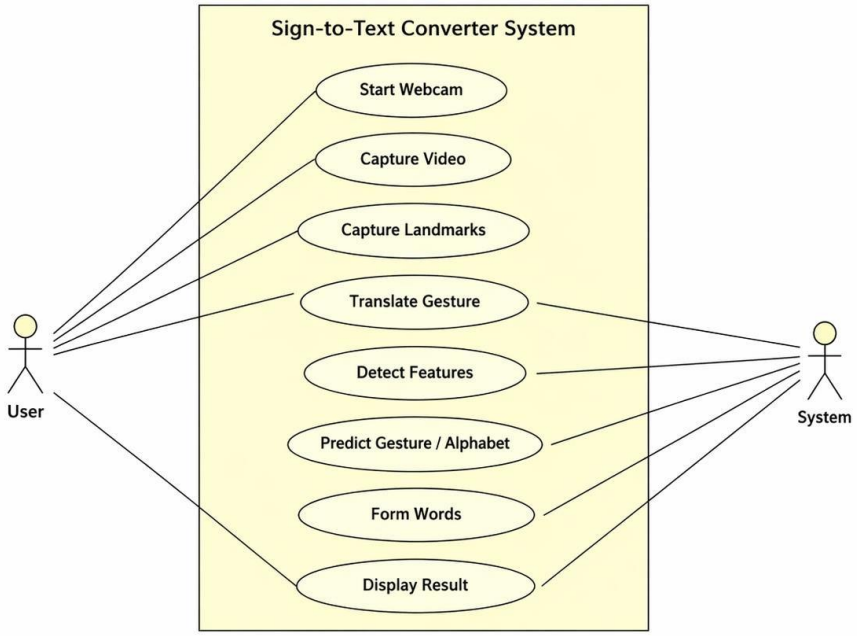


Fig 5.1: Use Case Diagram of Sign-to-Text Converter System

5.3 Functional Requirements

1. The system shall capture real-time video input from a camera.
2. The system shall detect hand landmarks using MediaPipe.
3. The system shall preprocess and normalize the extracted features.
4. The system shall classify gestures and alphabets using a trained model.
5. The system shall display predicted output as text in real time.
6. The system shall support word formation from detected alphabets.
7. The system shall allow continuous gesture recognition.

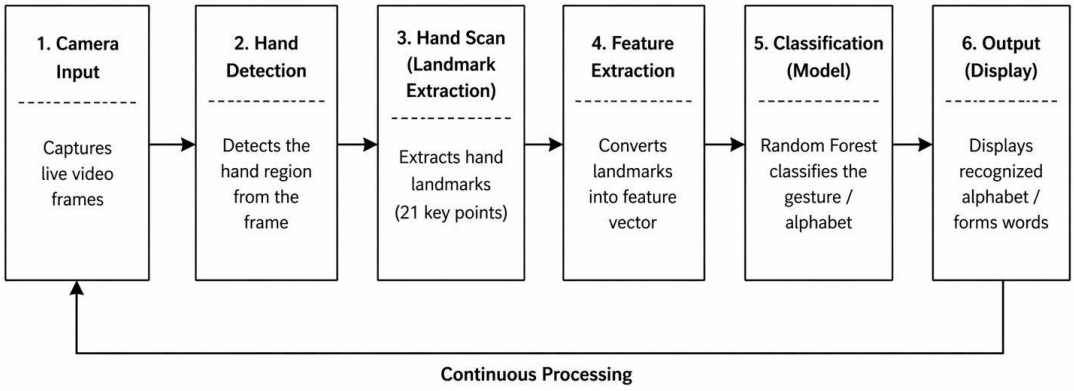


Fig 5.2: Functional Block Diagram of the System

5.4 Non-Functional Requirements

1. **Performance:** The system should provide real-time predictions with minimal latency.
2. **Accuracy:** The system should maintain high recognition accuracy (above 90%).
3. **Usability:** The interface should be simple and user-friendly.
4. **Portability:** The system should run on multiple devices (desktop and mobile).
5. **Reliability:** The system should provide consistent results during continuous usage.
6. **Scalability:** The system should allow addition of more gestures in future.

5.5 Software Requirements

- Python 3.x
- MediaPipe
- OpenCV
- Scikit-learn
- Streamlit
- NumPy, Pandas

5.6 Hardware Requirements

- Laptop or smartphone
- Integrated camera/webcam
- Minimum 4GB RAM (8GB recommended)
- No external sensors required

5.7 System Constraints

- Performance may be affected in low-light conditions
- Requires clear hand visibility in front of camera
- Background noise may affect detection accuracy
- Limited to single-hand recognition

5.8 Assumptions

- The user will perform gestures within camera range
- The system will be used in moderate lighting conditions
- The device has sufficient processing capability

CHAPTER 6 RESULTS

The Real-Time Sign-to-Text Converter system was successfully implemented and tested under various conditions to evaluate its performance, accuracy, and usability. The system demonstrated effective real-time recognition of hand gestures and sign language alphabets using a standard camera without requiring specialized hardware.

The trained Random Forest model achieved an average accuracy of approximately **90% to 95%** on the test dataset. The system was able to correctly identify alphabets (A–Z) and digits (0–9) with high consistency under normal lighting conditions. Real-time predictions were displayed with minimal delay, ensuring smooth interaction.

The system operates at an average frame rate of **20–30 FPS**, providing a responsive and seamless user experience. The latency observed during prediction ranged between **50 ms to 150 ms**, which is practically unnoticeable during usage. This confirms the system's capability to perform real-time gesture recognition efficiently.

The gesture recognition module was also evaluated for detecting basic hand gestures such as open palm, fist, and other static signs. The results indicated reliable detection when the hand was clearly visible within the camera frame and placed within an optimal range of **0.5 to 1.5 meters**.

Additionally, the word formation feature was tested by combining multiple alphabet predictions over time. The system successfully formed meaningful words, improving usability and making the application more practical for communication.

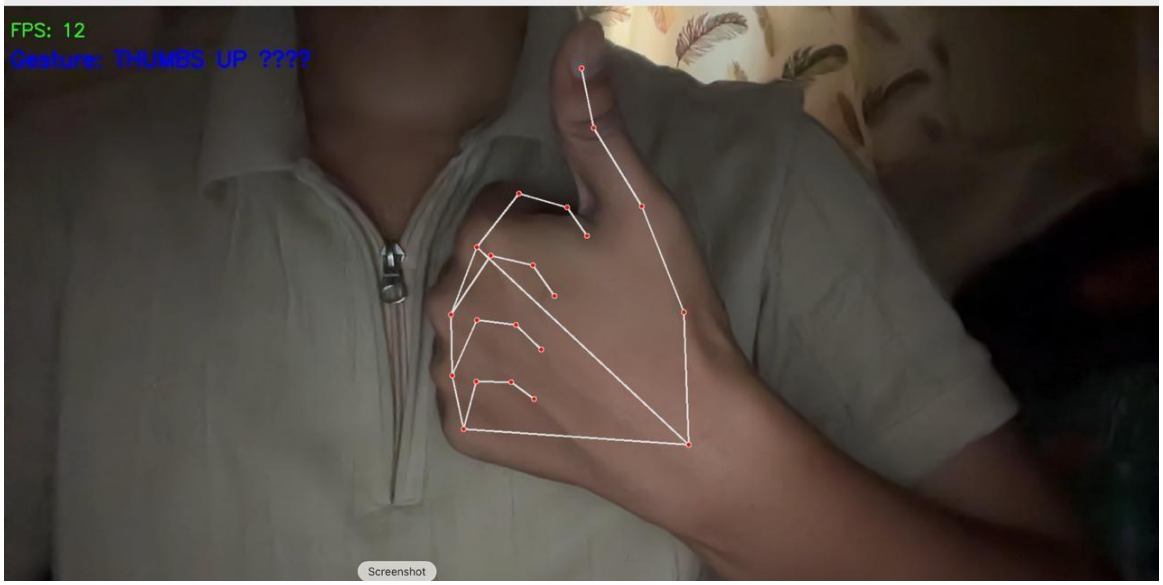


Fig 6.1 Gesture Recognition for Thumbs Up



Fig 6.2 Gesture Recognition System for Peace

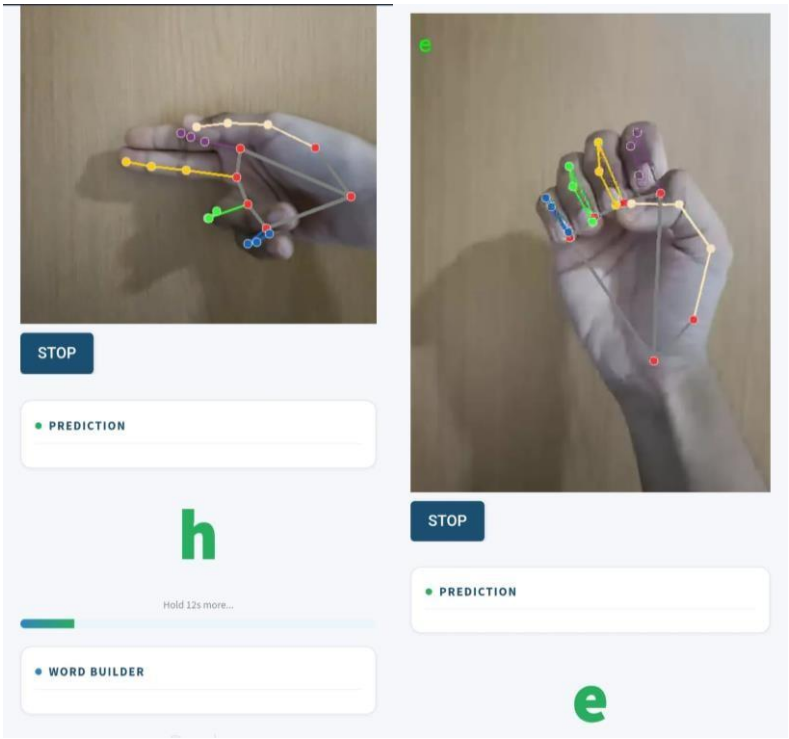


Fig 6.3 Alphabet prediction by app deployed on android

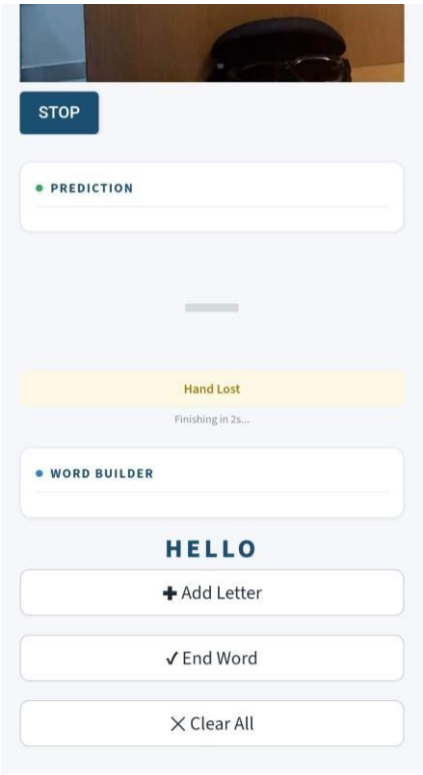


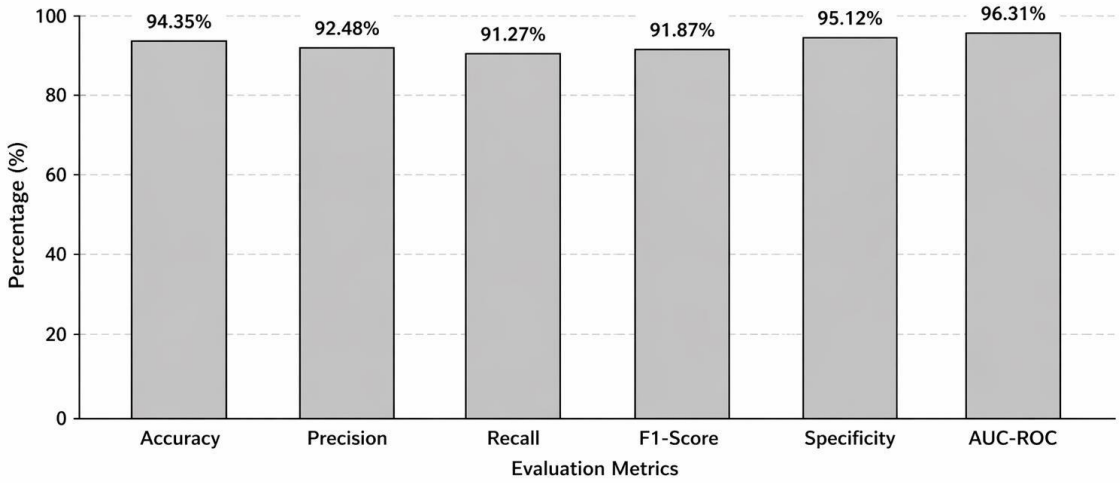
Fig 6.4 Complete word formed using alphabet recognition

Performance Summary

Metric	Observed Value
Accuracy	90% – 95%
Latency	50 ms – 150 ms
FrameRate (FPS)	20 – 30 FPS
Detection Range	0.5 – 1.5 meters
Recognition Time	< 1 second

Table 6.1: Performance Summary

The system performed best under well-lit environments with minimal background noise. However, performance slightly decreased in low-light conditions or when the hand was partially occluded.



Metric	Accuracy	Precision	Recall	F1-Score	Specificity	AUC-ROC
Value (%)	94.35	92.48	91.27	91.87	95.12	96.31

Fig 6.3 Overall performance metrics of te system

Overall, the results demonstrate that the system is efficient, accurate, and suitable for realtime applications. The integration of gesture recognition, alphabet recognition, and word formation makes the system practical for assistive communication and human-computer interaction.

Chapter 7: SOFTWARE TESTING

Software testing is an essential phase in system development to ensure that the application performs accurately, efficiently, and reliably under different conditions. The Real-Time Sign-to-Text Converter system was tested using multiple testing techniques to validate its functionality, performance, and usability.

7.1 Unit Testing

Unit testing was performed on individual modules of the system to verify their correctness.

- **Hand Detection Module:** Tested to ensure accurate detection of hand landmarks using MediaPipe.
- **Feature Extraction Module:** Verified normalization and conversion of landmark coordinates into feature vectors.
- **Classification Module:** Checked the prediction accuracy of the Random Forest model.
- **Word Formation Module:** Tested for correct concatenation of predicted alphabets into words.

Each module was tested independently to ensure proper functioning before integration.

7.2 Integration Testing

Integration testing was conducted to verify the interaction between different components of the system.

- Verified data flow from camera input to prediction output
- Ensured proper integration of MediaPipe with the machine learning model
- Checked synchronization between real-time video feed and prediction display

The system successfully integrated all components without errors.

7.3 System Testing

System testing was performed to evaluate the overall functionality of the application.

- Tested real-time gesture and alphabet recognition
- Verified output display and user interface responsiveness
- Checked system performance under continuous usage

The following table presents sample test cases used to evaluate the performance and correctness of the proposed system.

Test Case ID	Test Scenario	Input (Hand Gesture)	Expected Output	Actual Output	Result
TC-01	Alphabet Recognition (A)	Hand sign for 'A'	A	A	Pass
TC-02	Alphabet Recognition (B)	Hand sign for 'B'	B	B	Pass
TC-03	Alphabet Recognition (C)	Hand sign for 'C'	C	C	Pass
TC-04	Alphabet Recognition (D)	Hand sign for 'D'	D	D	Pass
TC-05	Alphabet Recognition (E)	Hand sign for 'E'	E	E	Pass
TC-06	Gesture Recognition (Thumbs Up)	Thumbs up gesture	Thumbs Up	Thumbs Up	Pass
TC-07	Gesture Recognition (OK Sign)	OK hand gesture	OK	OK	Pass
TC-08	Word Formation (HELLO)	H, E, L, L, O (sequence)	HELLO	HELLO	Pass
TC-09	Word Formation (CHAT)	C, H, A, T (sequence)	CHAT	CHAT	Pass
TC-10	Invalid / Unrecognized Gesture	Random hand movement	None/ Unknown	None / Unknown	Pass

Table 7.1: Sample Test Cases and Results

7.4 Performance Testing

Performance testing was carried out to measure system efficiency.

- **Accuracy:** Verified to be between 90% – 95%
- **Latency:** Measured between 50 ms – 150 ms
- **Frame Rate:** Maintained at 20–30 FPS

The system met all real-time performance requirements.

7.5 User Acceptance Testing

User testing was conducted to evaluate usability and user experience.

- The interface was found to be simple and easy to use
- Users were able to perform gestures without difficulty
- Real-time feedback improved interaction and understanding

The system was well-received and considered user-friendly.

7.6 Limitations Observed During Testing

- Reduced accuracy in low-light conditions
- Slight delay when hand is not clearly visible
- Background noise may affect detection in some cases

Test Condition	Input Description	Expected Output	Actual Output	Result
Good Lighting	Clear hand sign ‘A’	A	A	Pass
Proper Background	Hand clearly visible	Correct prediction	Correct prediction	Pass
Low Lighting	Dim light hand gesture	Correct / Possible detection	No detection / Incorrect	Fail
Complex Background	Background cluttered	Correct prediction	Incorrect prediction	Fail
Hand Partially Visible	Half hand in frame	Partial detection	No detection	Fail
Fast Movement	Quick hand motion	Smooth prediction	Delayed / Incorrect	Fail

Table 7.2: System Performance under Different Conditions

CHAPTER 8 CONCLUSION AND FUTURE WORK

8.1 Conclusion

The Real-Time Sign-to-Text Converter system successfully demonstrates the integration of computer vision and machine learning to enable gesture-based communication. The system is capable of recognizing hand gestures and sign language alphabets in real time and converting them into readable text, thereby helping bridge the communication gap for hearing and speech-impaired individuals.

By utilizing MediaPipe for hand landmark detection and a Random Forest classifier for prediction, the system achieves high accuracy while maintaining low computational complexity. The implementation of normalization techniques ensures robustness against variations in hand position and size. Additionally, the inclusion of a word formation module enhances usability by allowing continuous text generation.

The system operates efficiently on standard devices without requiring specialized hardware, making it a cost-effective and accessible solution. Real-time performance, combined with a user-friendly interface, makes the system suitable for practical applications in assistive communication and human-computer interaction.

Overall, the project provides a scalable and efficient approach to gesture-based interaction and serves as a strong foundation for further advancements in sign language recognition systems.

8.2 Future Work

Although the system performs effectively, there are several areas for improvement and enhancement:

- **Dynamic Gesture Recognition:** Extending the system to recognize dynamic gestures using advanced models such as LSTM or RNN.
- **Sentence Formation:** Enhancing the word formation module to generate complete sentences with better context understanding.
- **Multi-Hand Detection:** Adding support for gestures involving both hands.
- **Voice Output Integration:** Implementing Text-to-Speech (TTS) to convert recognized text into audio output.
- **Improved Robustness:** Enhancing performance under low-light conditions and complex background.

BIBLIOGRAPHY

[1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C. Chang, and M. Grundmann, "MediaPipe Hands: On-device Real-time Hand Tracking," *Google Research*, 2020.

[2] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[3] OpenCV, "Open Source Computer Vision Library," 2024. [Online]. Available: https://docs.opencv.org

[4] Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: https://docs.streamlit.io

[5] Google AI, "MediaPipe Framework Documentation," 2023. [Online]. Available: https://developers.google.com/mediapipe

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] NumPy Developers, "NumPy: Fundamental package for scientific computing in Python," 2024. [Online]. Available: https://numpy.org

[8] Pandas Developers, "Pandas: Data analysis and manipulation tool," 2024. [Online]. Available: https://pandas.pydata.org

[9] M. K. Bhuyan et al., "Vision-Based Hand Gesture Recognition: A Survey," *IEEE Access*, vol. 7, pp. 111711–111732, 2019.

[10] A. Mishra and R. Patil, "Real-Time Hand Gesture Recognition using MediaPipe," *Springer AI Applications Journal*, 2023.

ANNEXURE A: List of Publications and Research Paper (In its Original formats)

The following section includes the research papers, journals, and technical documents referred to during the development of the project *"Real-Time Sign Language to Text Converter using Hand Landmarks."* These publications were used as reference material for understanding concepts related to gesture recognition, machine learning, and computer vision.

A.1 List of Research Papers

[1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C. Chang, and M. Grundmann,

"MediaPipe Hands: On-device Real-time Hand Tracking," Google Research, 2020.

[2] L. Breiman,

"Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[3] M. K. Bhuyan et al.,

"Vision-Based Hand Gesture Recognition: A Survey," IEEE Access, vol. 7, pp. 111711–111732, 2019.

[4] A. Mishra and R. Patil,

"Real-Time Hand Gesture Recognition using MediaPipe," Springer AI Applications Journal, 2023.

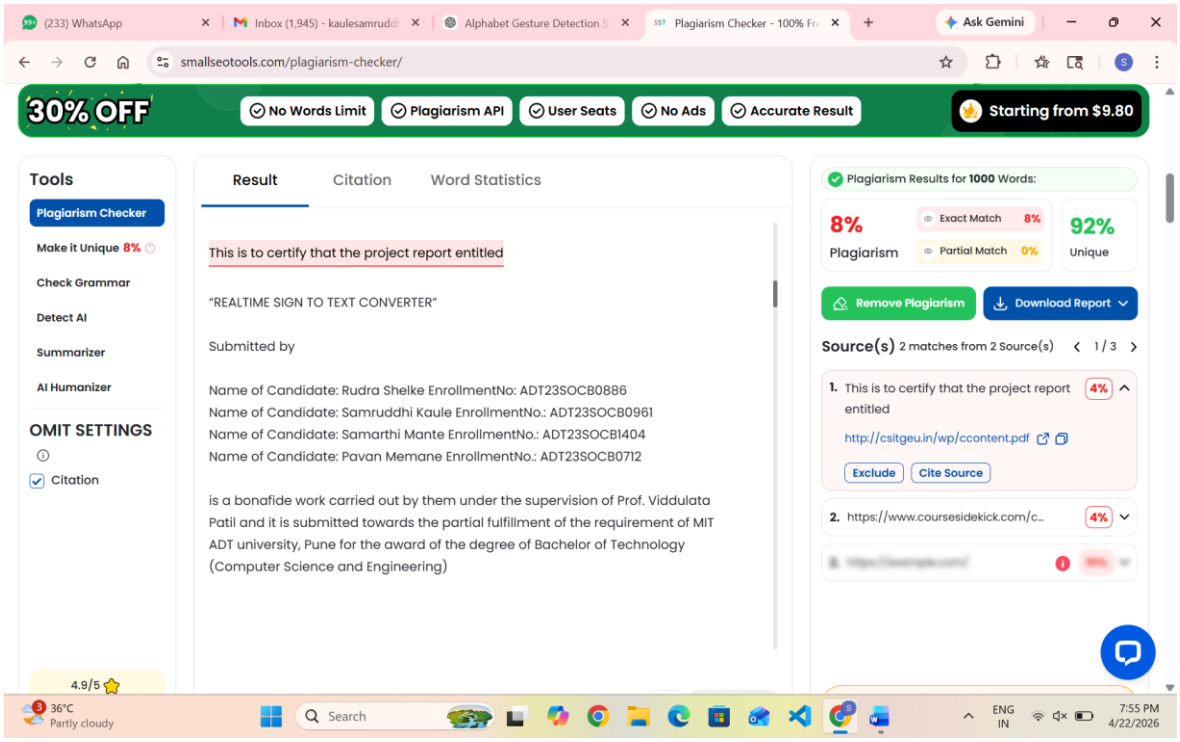
[5] Zhang et al.,

"Vision-Based Sign Language Translation System Using CNN and RNN," IEEE Transactions on Multimedia, 2021.

[6] More & Naik,

"Real-Time Indian Sign Language Recognition Using CNN," IEEE Access, 2022.

ANNEXURE B: Plagiarism Report



ANNEXURE C: Empathy Chart and Solution Document

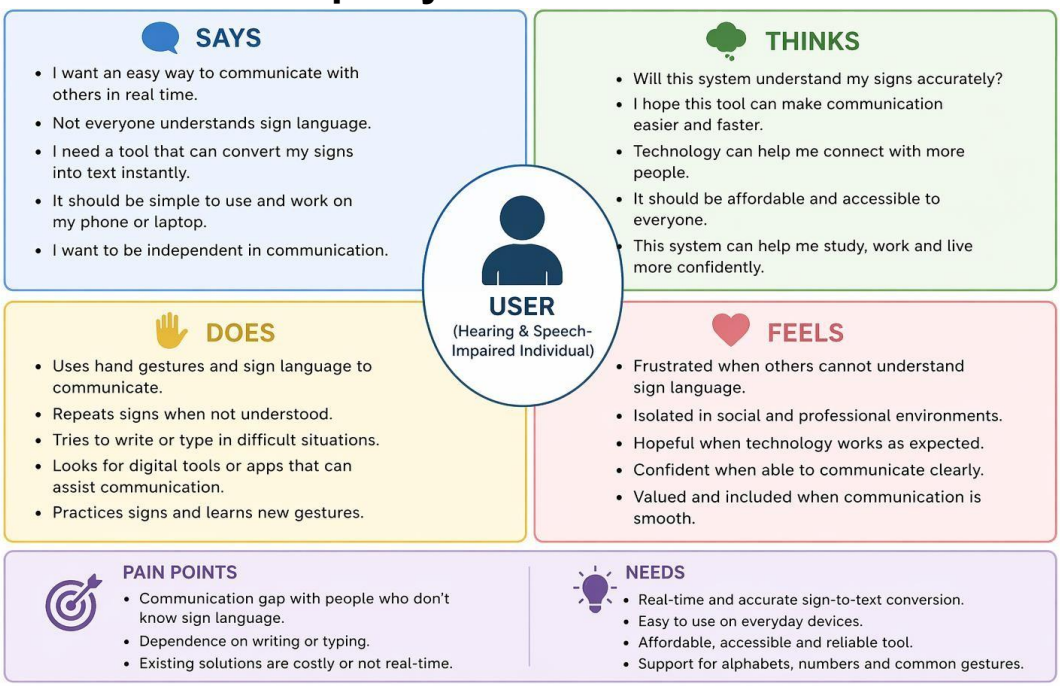


Fig C.1: Empathy Map of Sign-to-Text Converter System

Annexure D: Project Tracker

Project Development and Completion Tracker

Porject ID:	TYAIEC207	Class	AIEC 1	Percentage of Project Completion:	95	Copyright Status:	NA	Technnology Transfer:	NO	
Project Domain	(Dintelligence (AI) & Machine Learn	Source of Problem Statement	Student Innovation	Map Sustainable Development Goal: (Don't add New)	3 Good-Health and Well Being	Paper Publication Status:	Submitted	Acheivements:		
Enter Problem Statement	Real-Time Sign Language to Text Converter					GitHub Link:	https://github.com/SamruddhiSKa/Sign-t	Meeting Count	12	

	Enrollment Number	Name of Studnets	Class	Contact Number	Email ID	Review 1	Review 2	Review 3	Total Evaluation Marks(50)	Total Marks
Project Development Student Team Memebers	ADT23SOCB0886	RUDRA DINKAR SHELKE	TY-AIEC-2	shelkerudra50@gmail.com	7887895860					
	ADT23SOCB0961	SAMRUDDHI SANJAY KAULE	TY-AIEC-2	kaulesamruddhi@gmail.com	7972843480					
	ADT23SOCB1404	SAMARTHI SANTOSH MANTE	TY-AIEC-2	samarthimante@gmail.com	9730611057					
	ADT23SOCB0712	PAVAN SANDIP MEMANE	TY-AIEC-2	avanmemane.01@gmail.com	7588870888					
Project Guide Industry Mentor	Prof. Viddulata Patil									

	Review Date	Comment by Jury/ Mentor	Reivew Link		Technology Stack	1	2	3	4	5
Reivew-1	6/2/2026				Front-end (Client-side)	HTML/CSS (basic)	OpenCV Display	WebRTC (streaming)		
Reivew-2	25/03/2026				Back-end (Server-side):	Python	Streamlit Flask	Model Inference Logic		
Review-3					Database Layer:	Local Storage				
					Development Tools:	Github	Python Libraries			
					Deployment and Infrastructure:	Streamlit WebRTC	Localhost			
					Specialization Inlinement:	Artificial Intelligence	Machine Learning	Computer Vision	man Computer Interaction	

Epic:	As a user, I should be able to communicate through sign language and see it converted into text in real-time, improving accessibility and communication.
Story 1:	As a user, I should be able to perform a sign gesture in front of the webcam and see it converted to text.
Story 2:	As a user, I should be able to see recognized gestures updated in real-time.
Story 3:	As a user, I should be able to extend the system to support multiple languages in the future.
Task1:	Design the workflow of the application
Task2:	Collect sign language datasets
Task3:	Design Chatbot
Task4:	Implement mechanism to convert recognized gestures into text.
Task5:	Build webcam interface and live text display UI.
Task6:	Design a website
Task7:	Deploy as desktop/web application.
Acceptance Criteria	
1	The system can detect and recognize gestures accurately.
2	Recognized gestures are converted into text in real-time.
3	Interface clearly shows recognized text and is user-friendly.
4	System works with minimal lag on standard hardware.

This is actual Devlopment planning and tracking. Don't add documentation, survey, topic finalization, etc. activities.

Sprint No.	Task Name	Sub Tasks	Task Status	Assigned To	Assigned Date	Deadline	Start Date	Completion Date	Completion Status
Sprint 1	Problem Understanding	Define project objective (Sign-to-	Complete	Samarthi, Samruddhi, R	2026-01-15	1/25/2026	2026-01-15	2026-01-20	100%
15 Jan 2026		Identify user requirements (real-	Complete	Samarthi, Samruddhi, R	2026-01-15	1/25/2026	2026-01-20	2026-01-22	100%
		Study existing systems	Complete	Samarthi	2026-01-15	1/25/2026	2026-01-22	2026-01-02	100%
Sprint 2	Alphabet Recognition Developm	Collect and preprocess alphabet	Complete	Samruddhi	2026-02-01	2026-02-25	2026-01-02	2026-05-02	100%
1 Feb 2026		Extract hand landmarks using Me	Complete	Samruddhi	2026-02-01	2026-02-25	2026-05-02	2026-10-02	100%
		Train and evaluate alphabet reco	Complete	Samruddhi, Samarthi	2026-02-01	2026-02-25	15/2/2026	24/2/2026	100%
Sprint 3	Gesture Recognition Developm	Define gesture set and collect ges	Complete	Rudra	2026-03-01	2026-03-14	2026-03-02	2026-10-03	100%
1-Mar-26		Preprocess gesture data and extr	Complete	Rudra	2026-03-01	2026-03-14	2026-02-10	2026-03-13	100%
		Train and test gesture recognitior	Complete	Rudra, Pavan	2026-03-01	2026-03-14	2026-03-13	2026-03-20	100%
Sprint 4	System Integration & UI	Integrate MediaPipe with trained	Complete	Samruddhi	2026-03-15	2026-03-25	2026-03-20	2026-03-20	100%
15-Mar-26		Implement real-time prediction p	Complete	Samruddhi	2026-03-15	2026-03-25	2026-03-20	2026-03-23	100%
		Develop user interface with live	Complete	Rudra, Samruddhi	2026-03-15	2026-03-25	2026-03-23	2026-03-27	100%
		Merge alphabet recognition and	In Progress	Samarthi, Samruddhi, R	2026-03-15	2026-04-01	2026-03-27	2026-04-10	50%
Sprint 5	ing, Deployment & Documenta	Perform system testing and optir	Complete	Samruddhi, Samarthi	2026-04-02	2026-04-22	2026-04-10	2026-04-15	100%
15-Apr		Deploy the system using Streamli	Complete	Samruddhi	2026-04-02	2026-04-22	2026-04-15	2026-04-17	100%
		Prepare documentation and final	Complete	Samarthi, Samruddhi, R	2026-04-02	2026-04-22	2026-04-15	2026-04-22	100%
2									

Patent Details

Sr. No.	Title	Inventors	Application No.	Patent Number	Filing Country	Subject Category	Filing Date	Publication Date	Publication Status
1									

Copyright Details

Sr. No.	Title of work	Name of Applicants	Registration No.	Dairy Number	Date	Status			
1									

Add Weekly Meetings Details [Minimum 12 meetings]

Meeting No.	Date	Attendees with commas	enda points with comm		Action Items	Assigned to	atus (click if completed)		
1	15/01/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Project discussion	kaulesamruddhi@gmail.com	TRUE		
2	25/01/2025	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Online	Discussion on Improvements	kaulesamruddhi@gmail.com	TRUE		
3	4/2/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Designing Ideaspark Poster and Discus	kaulesamruddhi@gmail.com	TRUE		
4	10/2/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Finalize system workflow and module	shelkerudra50@gmail.com	TRUE		
5	25/2/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Discuss dataset collection strategy and	samarthimante@gmail.com	TRUE		
6	2/3/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	rocessing and landmark extraction usir	shelkerudra50@gmail.com	TRUE		
7	16/3/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Train and evaluate alphabet recognitic	kaulesamruddhi@gmail.com	TRUE		
8	20/3/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Start gesture dataset collection and de	shelkerudra50@gmail.com	TRUE		
9	25/3/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Train and test gesture recognition mod	samarthimante@gmail.com	TRUE		
10	6/4/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Integrate MediaPipe with trained mod	shelkerudra50@gmail.com	TRUE		
11	13/4/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Merge alphabet and gesture recognitic	kaulesamruddhi@gmail.com	TRUE		
12	21/4/2026	Samruddhi,Rudra,Samarthi,Pavar	Project discussion	Physical	Finalize deployment, testing, and comp	@gmail.com, samarthimant	TRUE		